Semana 27 — Conteúdo de 2 aulas

Banco de Dados

Ricardo Pasquati Pontarolli

Técnico em Redes de Computadores · IFSP Boituva · pasquati@ifsp.edu.br · v9

Semana 27 — Conteúdo de 2 aulas

- **Aula 43:** Funções de String e de Data
- **Aula 44:** INSERT: inserindo dados

Aula 43 — Funções de String e de Data

Objetivos da Aula

- Conhecer **funções de texto**.
- Conhecer **funções de data**.
- Manipular valores nas consultas.

Funções de String

- **UPPER()** / **LOWER()** — maiúsc./minúsc.
- **CONCAT()** — junta textos.
- **LENGTH()** — tamanho.
- **SUBSTRING()** — parte do texto.
- **TRIM()** — remove espaços.



Exemplos de String

```sql
SELECT UPPER(nome) FROM Aluno;
SELECT CONCAT(nome, ' - ', cidade) FROM Aluno;
SELECT LENGTH(nome) FROM Aluno;
```

Funções de Data

- **CURDATE()** — data atual.
- **NOW()** — data e hora.
- **YEAR()**, **MONTH()**, **DAY()**.
- **DATEDIFF()** — diferença de dias.

Exemplos de Data

```sql
SELECT CURDATE();
SELECT YEAR(data_pedido) FROM Pedido;
SELECT DATEDIFF(CURDATE(), data_nasc) FROM Aluno;
```

Calculando Idade

```sql
SELECT nome,
  TIMESTAMPDIFF(YEAR, data_nasc, CURDATE()) AS idade
FROM Aluno;
```

Note

As funções permitem **calcular e formatar** direto na consulta, sem alterar os dados armazenados.

Prática

- Mostre os nomes em **maiúsculas**.
- Calcule a **idade** dos alunos.

Resumo da Aula

- **String:** UPPER, CONCAT, LENGTH...
- **Data:** CURDATE, YEAR, DATEDIFF...
- Enriqueceram as **consultas**.

Referências

- Documentação oficial do MySQL.

Aula 44 — INSERT: inserindo dados

Objetivos da Aula

- Compreender o comando **INSERT**.
- Inserir **um registro**.
- Conhecer boas práticas de inserção.

O comando INSERT

- Adiciona **linhas** a uma tabela.
- Parte da **DML**.
- Trabalha sobre tabela **existente**.

Sintaxe

```sql
INSERT INTO Aluno (nome, idade, cidade)
VALUES ('Ana', 16, 'Boituva');
```

- Colunas **entre parênteses**.
- Valores na **mesma ordem**.



Sem Listar Colunas

```sql
INSERT INTO Aluno VALUES (NULL, 'Ana', 16, 'Boituva');
```

- Precisa seguir **todas** as colunas na ordem.
- Menos **seguro** — evite!

Inserindo com AUTO_INCREMENT

```sql
INSERT INTO Aluno (nome, idade) VALUES ('Bia', 17);
```

- O `id` é **gerado** automaticamente.
- Não o informamos.

Conferindo

```
SELECT * FROM Aluno;
```

- Verifique se entrou **certo**.

> **Note**
>
> Sempre liste as **colunas** no INSERT: se a tabela mudar, seu comando continua funcionando.

Cuidados

- Respeitar **tipos** de dados.
- Aspas em **textos**.
- FK precisa **existir**.
- Não violar **restrições** (NOT NULL, UNIQUE).

Prática

- Insira 3 alunos.
- Confira com SELECT.

Resumo da Aula

- **INSERT** adiciona registros.
- Melhor **listar as colunas**.
- Respeitar **tipos e restrições**.

Referências

- ROB, P.; CORONEL, C. *Sistemas de Banco de Dados*. Cengage.

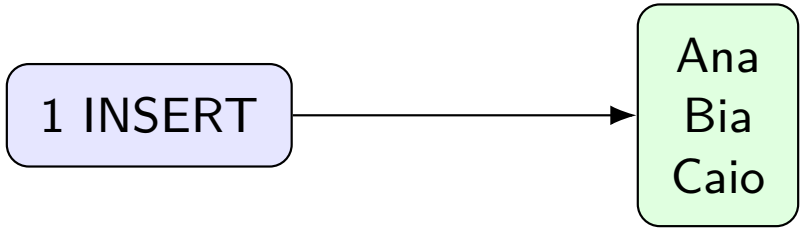
Objetivos da Aula

- Inserir **várias linhas** de uma vez.
- Usar **valores padrão**.
- Inserir dados a partir de outra **consulta**.

Várias Linhas

```sql
INSERT INTO Aluno (nome, idade) VALUES
('Ana', 16),
('Bia', 17),
('Caio', 15);
```

- Um só comando, **vários registros**.
- Mais **rápido** que vários INSERTs.



Valores Padrão (DEFAULT)

- Colunas podem ter valor **padrão**.
- `DEFAULT` define o valor automático.
- Se não informar, o padrão é usado.

Inserindo com DEFAULT

```
INSERT INTO Aluno (nome) VALUES ('Duda');
```

- Só o nome; o resto usa o **padrão** (ou fica nulo).

INSERT ... SELECT

```
INSERT INTO AlunosAntigos (nome, idade)
SELECT nome, idade FROM Aluno WHERE idade > 18;
```

- Copia dados de outra **consulta**.
- Útil para **migração** e cópias.

Ignorando Erros

- `INSERT IGNORE` pula violações.
- Use com **cautela**.

> **Note**
>
> O `INSERT ... SELECT` é a forma elegante de **copiar** dados entre tabelas sem sair do banco.

Prática

- Insira vários alunos de uma vez.
- Crie uma tabela e copie dados com `INSERT ... SELECT`.

Resumo da Aula

- INSERT pode ter **várias linhas**.
- `DEFAULT` preenche automaticamente.
- `INSERT ... SELECT` copia de uma consulta.

Referências

- Documentação oficial do MySQL.